

Bologna: Block-Based 802.11 Transmission Recovery

Bo Han
Department of Computer Science
University of Maryland
College Park, MD 20742, U.S.

Francesco Gringoli and Luca Cominardi
DEA - Facoltà di Ingegneria
Università degli Studi di Brescia
25123 Brescia, Italy

ABSTRACT

In this paper, we describe the design, implementation, and evaluation of Bologna, a practical block-based IEEE 802.11 data transmission recovery protocol. Bologna is built on a well-known recovery scheme that identifies corrupted bit blocks by performing per-block checksums and then requests the retransmission of only the error blocks. The two key challenges for Bologna design are (1) to find a proper checksum which is strong enough for error detection and meanwhile lightweight in computation and communication overhead, and (2) to modify the IEEE 802.11 standard as little as possible and make the implementation compatible with existing 802.11 devices. We have chosen Fletcher-16 as the checksum scheme and implemented Bologna using an open source firmware. We also deployed Bologna in our testbed and compared its performance to 802.11.

Categories and Subject Descriptors

C.2.1 [Computer Communication Network]: Network Architecture and Design—*Wireless communication*

General Terms

Design, Measurement, Performance

Keywords

Partial packet recovery; block-based recovery; Fletcher-16; 802.11; compatibility; implementation.

1. INTRODUCTION

Communication error recovery has been a key research topic since the very beginning of data networking, especially for wireless LANs due to their high bit error rate. Traditionally, there are two groups of transmission error recovery algorithms: the ARQ (Automatic Repeat reQuest) systems and the FEC (Forward Error Correction) systems. A variation of ARQ, called Hybrid-ARQ, combines the basic ARQ with FEC by replacing the error detection bits embedded in data packets with error correction bits. Several protocols

have been proposed based on these basic approaches. For example, PPR [1] is an ARQ-based protocol which benefits from physical layer symbol quality information and only requests likely corrupted symbols. ZipTx [2] is a Hybrid-ARQ-based protocol which retransmits error correction bits to repair corrupted packets. Compared to 802.11 MAC protocol which retransmits full packets, these protocols have confirmed the effectiveness of partial packet recovery.

Motivated by the success of these methods, we have designed a block-based partial packet recovery protocol for 802.11, called Maranello [3]. In this paper, we continue the effort of making Maranello more friendly towards other 802.11 devices and propose a new recovery protocol, Bologna. In Bologna, we followed a well-known practice that embeds per-block checksums in data packets, identifies error blocks in corrupted packets through these checksums, and then retransmits only the blocks with errors. A key challenge of designing a practical recovery protocol for 802.11 is to make as little as possible changes to the 802.11 standard. Bologna has the advantage of extending the 802.11 ACK frame with a 3-byte only error-block bitmap and thus can coexist with other 802.11 devices. When errors are detected in a received packet, a Bologna-receiver exploits the reserved ACK airtime to send back to the sender a negative acknowledgement (NACK) with the bitmap embedded and thus can efficiently utilize the scarce channel resource.

We have made the following contributions in this paper.

- We propose to use Fletcher-16 checksum for Bologna, due to its known low overhead, and evaluate its error-detection capability. The experiment results show that it can detect almost all the error blocks, even for the worst-case scenario with very high bit error rate.
- Based on our Bologna implementation in an open source firmware and its Linux driver, we compare its performance to unmodified 802.11 on an indoor testbed. The preliminary experiment results show that Bologna can increase the UDP throughput of 802.11.

In what follows, we provide an overview of existing recovery protocols in Section 2 and present the Bologna design in Section 3. In Section 4, we implement Bologna in an open source firmware, and evaluate its performance on our testbed. Finally, we conclude in Section 5.

2. RELATED WORK

In this section, we briefly review the existing recovery protocols and compare Bologna to them. We refer interested readers to Han et al. [3] for a detailed literature review.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

S3'10, September 20, 2010, Chicago, Illinois, USA.

Copyright 2010 ACM 978-1-4503-0144-2/10/09 ...\$10.00.

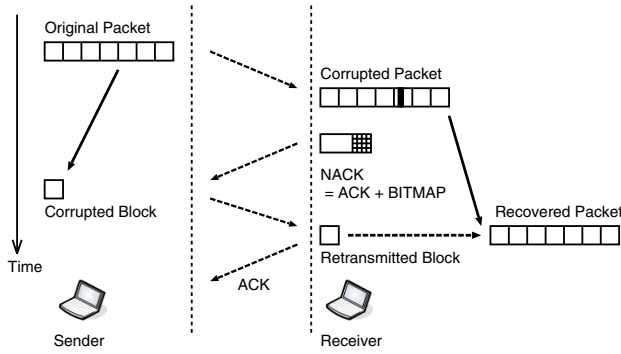


Figure 1: Schematic diagram of Bologna.

We first discuss two ARQ-based approaches: PPR [1] and Maranello [3]. PPR benefits from physical layer symbol quality information and thus can request retransmissions of only those likely corrupted symbols. However, most commercial 802.11 cards do not provide such information to higher layers. In Maranello, the receiver computes per-block checksums for corrupt packets and sends them back as a NACK frame. The sender then retransmits only those error blocks identified by the checksums. Although Maranello introduces no extra bits for correct data packets, its NACK frame is around 100 bytes larger than an 802.11 ACK frame. Because the transmitter does not know before hand whether the receiver will send an ACK or a NACK, it cannot estimate exactly how much time is needed to complete the DATA-ACK/NACK handshake. As a result, Maranello may reduce the effectiveness of the virtual carrier sensing mechanism of 802.11. As we will see shortly, Bologna addresses these issues to become more practical and more friendly towards coexisting 802.11 devices.

We then review a Hybrid-ARQ-based scheme, ZipTx [2], which is a two-round recovery protocol. Based on the estimated bit error rate from the receiver, a ZipTx-sender transmits only a small number of error recovery bits for a corrupted packet in the first round. If the receiver cannot repair the packet using these bits, the sender transmits more bits in the second round. The sender retransmits the whole packet when both rounds fail. Compared to Bologna, ZipTx may increase recovery latency, because it disables retransmissions at MAC layer and sends its own ACKs for a group of accumulated packets.

3. Bologna DESIGN

We present an overview of Bologna design and evaluate several error-detection schemes in this section.

3.1 Overview

We have shown in Maranello [3] that error bits within corrupted packets are usually clustered and thus block-based recovery has the potential to increase the link throughput by retransmitting only corrupt blocks. Although Maranello has several advantages, its large NACK frame may cause collisions with data packets from hidden terminals and defer data transmission from other 802.11 stations. Bologna can solve these problems by further exploring the design space.

Figure 1 presents an overview of Bologna design, which can be summarized as follows.

- (1) When a Bologna-sender transmits a data frame, it di-

vides the frame into k -byte blocks and embeds a separate checksum for each block.

- (2) After receiving a corrupt packet, a Bologna-receiver uses the embedded checksums to detect error blocks and constructs a bitmap indicating which blocks are corrupted. Then it replies to the sender a NACK which includes the bitmap. If no error is detected, the Bologna-receiver sends back a standard 802.11 ACK, and passes the received data packet to upper layers, after stripping off the embedded checksums.
- (3) After receiving a NACK, the transmitter sends a repair packet with only those error blocks identified by the bitmap in the NACK frame.
- (4) Once the repair packet is successfully received, the receiver sends an 802.11 ACK and combines the retransmitted blocks with the buffered corrupted packet to reconstruct the original data packet (again stripping off the checksums).

Bologna can solve the collision issue caused by Maranello, because its NACK frame is much shorter than Maranello and is only 3 bytes longer than an 802.11 ACK frame in our implementation. However, a limitation of Bologna is that it introduces overhead for all the data packets, no matter whether they are corrupted or not. We argue that, due to the 802.11 inter-frame space and protocol header overhead of various layers, the overhead of embedded checksums may be much lower than what we expected, if we can choose a proper checksum scheme. Since data packets may be transmitted at a higher data rate than ACK/NACK frames, embedding checksums in data packets also reduces the total exchange time than embedding them in NACK frames. Moreover, compared to Maranello, the shorter length of Bologna's NACK frame can increase its delivery probability and thus Bologna has the potential to recover more corrupted packets.

3.2 Which Checksum Scheme to Use?

We choose Fletcher-16 [4] to compute the per-block checksums for Bologna. Fletcher-16 is an arithmetic checksum and is able to detect all single bit errors, a single error block with length less than 16 bits, and all double bit errors separated by at most 16 bits [5]. We do not select CRC checksums because they are not computationally efficient for their software implementation. There is also a trade-off between the checksum overhead and protocol efficiency. Longer checksums are robust in terms of error detection, but introduce higher data overhead. Fletcher-16 represents a good compromise. In the following, we compare the error-detection capability of Fletcher-16 to CRC-32 and Fletcher-32 for all the 802.11 b/g data rates.

The experiment setup is as follows. We randomly choose a link from our testbed and reduce the transmission power to 1 dBm to induce more corrupt packets. During a single experiment, we greedily inject traffic by sending out 1,000,000 1200-byte packets. The block size is 64 bytes, including the checksums. The content of these packets is randomly generated, but is known to both the sender and receiver. Thus, we can calculate the number of corrupted blocks and undetected error blocks at the receiver side.

Table 1 presents these numbers, (undetected error blocks)/(corrupted blocks), for the three checksum schemes. For low data rates, the number of error blocks is small. One of the possible reasons is that their bit error rate is usually lower than that of high data rates, given the same link quality

Data Rate	CRC-32	Fletcher-16	Fletcher-32
1 Mbps	0/997	0/1221	0/1789
2 Mbps	0/5658	1/20675	0/7347
5.5 Mbps	0/13522	1/32129	0/8472
6 Mbps	0/25396	0/36820	0/19056
9 Mbps	0/29118	0/72622	0/31117
11 Mbps	0/22915	0/57628	0/18623
12 Mbps	0/24036	11/52913	0/26509
18 Mbps	0/64940	3/140466	0/54168
24 Mbps	0/166960	1/306489	0/151486
36 Mbps	0/822764	64/3767553	0/1308132
48 Mbps	0/3013719	38/1981888	0/3231432
54 Mbps	0/1071115	12/823488	0/1354104

Table 1: Performance evaluation of various checksums for different data rates.

(e.g., signal-to-noise-ratio). The results show that CRC-32 and Fletcher-32 can detect all the corrupt blocks. Although Fletcher-16 cannot detect all the bit errors, its performance is still acceptable, because the table shows the results for the worst-case scenario where the bit error rate of the selected link is very high. For example, when the data rate is 48 Mbps, all the 105,630 dumped packets were corrupted with an average of 18.76 error blocks per packet. In practice, the rate adaption protocol of 802.11 would not choose such a data rate. We also verified on our testbed that with rate adaption enabled the number of undetected error blocks is extremely low (less than 0.005%).

4. IMPLEMENTATION AND EVALUATION

In this section, we first present the details of Bologna implementation and then evaluate its performance.

4.1 Bologna Implementation

Although the well-known block-based recovery scheme has been proposed for several decades, there is no implementation and detailed performance evaluation on the widely deployed 802.11 platform. Usually ACK frames and retransmissions are handled by the firmware/hardware due to the strict timing requirement of 802.11. Given this constraint, we implement Bologna using OpenFWWF¹, an open source firmware for Broadcom chipsets and its Linux device driver, b43², similar to the Maranello implementation.

We implement the time-critical operations of Bologna, including NACK generation and recovery packet construction in OpenFWWF and other functions in the b43 driver. As in Maranello, a Bologna-receiver calculates block checksums for corrupted packets in the firmware and generates the error-block bitmap by comparing the computed checksums to the received ones. We use 64 bytes as the block size due to hardware limitations. The size of an 802.11 ACK frame is 14 bytes. By embedding the error-block bitmap, a Bologna NACK frame is at most 3 bytes longer than an 802.11 ACK frame (64-byte block size, 1-bit for each block and 24 blocks maximum for the typical MTU of a LAN). When a Bologna-sender gets a NACK, it constructs a recovery packet also in the firmware which includes only error blocks indicated by the NACK frame's bitmap. Other functions, including the pre-computation of block checksums (sender side) and

¹<http://www.ing.unibs.it/openfwf/>

²<http://linuxwireless.org/en/users/Drivers/b43/>

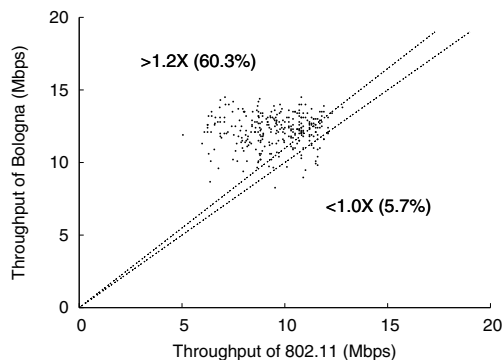


Figure 2: Bologna outperforms 802.11 in terms of UDP throughput.

the reconstruction of data packets (receiver side), are implemented in the driver, because they have no timing requirement and the host machine's CPU is much more powerful than the wireless card's microprocessor.

4.2 Bologna Evaluation

We compare the UDP throughput of Bologna to unmodified 802.11 on our testbed and show the results in Figure 2. We run 1-minute Iperf 600 times to generate UDP packets and saturate the wireless channel, using Bologna and 802.11 alternatively. In this plot, the x-axis presents the throughput of 802.11 and the y-axis is that of Bologna. The position of a point indicates the throughput gain. Figure 2 shows that Bologna can increase 802.11's throughput for UDP traffic often by 20% or more. We also compare the UDP throughput of Bologna to Maranello and the result shows that Bologna outperforms Maranello. We omit that plot due to space limitation.

5. CONCLUSION

In this paper, we describe the design, implementation and evaluation of a partial packet recovery protocol for 802.11, called Bologna. Our preliminary experiment results show that Bologna can improve the throughput of 802.11 for UDP traffic by up to 20%, on our testbed. We are currently optimizing the Bologna implementation and conducting a more detailed performance evaluation of it.

Acknowledgement: We thank Lusheng Ji for his encouragement, support and valuable inputs.

6. REFERENCES

- [1] K. Jamieson and H. Balakrishnan. PPR: Partial Packet Recovery for Wireless Networks. In *SIGCOMM*, pages 409–420, August 2007.
- [2] K. C.-J. Lin, N. Kushman, and D. Katabi. ZipTx: Harnessing Partial Packets in 802.11 Networks. In *MOBICOM*, pages 351–362, September 2008.
- [3] B. Han *et al.* Maranello: Practical Partial Packet Recovery for 802.11. In *NSDI*, April 2010.
- [4] J. G. Fletcher. An Arithmetic Checksum for Serial Transmissions. *IEEE Transactions on Communications*, 30(1):247–252, January 1982.
- [5] J. Stone *et al.* Performance of checksums and CRC's over real data. *IEEE/ACM Transactions on Networking*, 6(5):529–543, October 1998.